

Zadanie 5: programowanie genetyczne i regresja symboliczna

Autorzy: Anna Moroń, Kalina Rączka, Emilia Sarna

Termin realizacji: 18 maja 2026

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Zmodyfikuj przykład `pysr_demo.ipynb` tak, aby uczył się funkcji $f(x) = 2.2\sin(x_0 + 2x_1) - x_5^2 - 3$ której dziedziną jest $\mathbb{R}^6$. Uczenie ma się odbywać w oparciu o 200 wylosowanych z dziedziny próbek (między -5 a 5).
2. Zanotuj wzory trzech rozwiązań o najwyższej wartości `score` oraz rozwiązanie `best` dla następujących zestawów ustawień:
 1. `binary_operators=["+", "*"], unary_operators=["cos", "exp", "sin"], maxsize=20,`
 2. `binary_operators=["+", "*", "-", "^"], unary_operators=["cos", "exp", "sin", "log"], maxsize=30,` (dodaj ograniczenie dla argumentów operatora `^`: <https://astroautomata.com/PySR/v1.5.9/options.html#constraining-use-of-operators>).
 3. `binary_operators=["+", "*", "-", "^"], unary_operators=["exp", "sin"], maxsize=15.`
3. Powtórz eksperymenty z zadania na 3.0 po dodaniu szumu do próbek z funkcji f (rozkład normalny o średniej 0 i odchyleniu standardowym 0.5)

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Dodaj do porównania dopasowanie oparte o próbki losowane w szerszym zakresie (między -15 a 15) oraz wyższy poziom szumu (odchylenie standardowe równe 2 oraz 5).

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Zamień funkcję f na $f(x) = 2.2 \sin(x_0 + 2x_1) - x_5^2 - p(\lfloor x_0 \rfloor)$ gdzie $p(i)$ oznacza i -tą liczbę pierwszą. Uwzględnij `p` jako dodatkowy operator unarny analogicznie do przykładu “Julia packages and types” z notatnika `pysr_demo.ipynb`. Powtórz eksperymenty opisane w zadaniach na 3.0 i 4.0.

```
!pip install -U pysr
```

```
import pysr
import sympy
import numpy as np
import warnings
from matplotlib import pyplot as plt
from pysr import PySRRegressor
from sklearn.model_selection import train_test_split
```

```
import pandas as pd
from sklearn.metrics import r2_score
```

Funkcja której uczymy:

$f(x) = 2.2 \sin(x_0 + 2x_1) - x_5^2 - 3$, z dziedziną jest $\mathbb{R}^6$

Przygotowana próbka

Na potrzeby pierwszych testów przygotowano próbkę 200 wartości w dziedzinie $\mathbb{R}^6$, o wartościach między -5 a 5

```
np.random.seed(0)
X = 5 * np.random.randn(200, 6)
y = 2.2 * np.sin(X[:, 0] + 2 * X[:, 1]) - X[:, 5] ** 2 - 3
```

```
print(X)
```

```
[[ 8.82026173  2.00078604  4.89368992 11.204466   9.33778995
 -4.8863894 ]
 [ 4.75044209 -0.75678604 -0.51609426  2.05299251  0.72021786
  7.27136753]
 [ 3.80518863  0.60837508  2.21931616  1.66837164  7.47039537
 -1.02579132]
 ...
 [-1.50239393 -1.77514366 -9.46180947 -0.88906572  1.25499058
  5.27378963]
 [ 4.80023871 -2.08249541 -1.38411497  5.61952653 -0.86731949
 -2.5501477 ]
 [ 6.96259225  5.18792834  0.09395896 -2.96888724 -10.0594016
  2.94851803]]
```

```
default_pysr_params = dict(
    populations=30,
    model_selection="best",
    verbosity=0,
)
```

Modele

Przygotowano 3 modele o ustawieniach według polecenia: 1. binary_operators: ['+', '**'], unary_operators: ['cos', 'exp', 'sin'] 2. binary_operators: ['+', '**', '-', '^'], unary_operators: ['cos', 'exp', 'sin', 'log'] 3. binary_operators: ['+', '**', '-', '^'], unary_operators: ['exp', 'sin']

Do operatora $\wedge$ dodano ograniczenie, że drugim argumentem (wykładnikiem) musi być liczba - nie tworzymy funkcji wykładniczych.

Printujemy 3 najlepsze rozwiązania z każdego modelu o najlepszym score. Jako score użyta jest domyślna metryka określona wzorem:

$$\text{score} = -\log(\text{loss}_i / \text{loss}_{i-1}) / (\text{complexity}_i - \text{complexity}_{i-1}).$$

Dodatkowo wypisujemy rozwiązanie “best” zwracane domyślnie z modelu.

```
def run_all_setups(X, y):
    print("KONFIGURACJA 1")
    print("  binary_operators: ['+', '*'], unary_operators: ['cos', 'exp',
    ↪ 'sin']")

    model1 = PySRRegressor(
        binary_operators=["+", "*"],
        unary_operators=["cos", "exp", "sin"],
        maxsize=20,
        **default_pysr_params,
    )
    with warnings.catch_warnings():
        warnings.filterwarnings(
            "ignore",
            message=r"Note: it looks like you are running in Jupyter.*",
            category=UserWarning,
        )
        model1.fit(X, y)

    print("\nTop 3 rozwiązania (maksymalny score):")
    eq1 = model1.equations_
    top3_eq1 = eq1.nlargest(3, 'score')[['complexity', 'loss', 'score',
    ↪ 'equation']]
    for idx, row in top3_eq1.iterrows():
        print(f"      [{row['complexity']}] Score: {row['score']:.6f}, Loss:
        ↪ {row['loss']:.6f}")
        print(f"  Równanie: {row['equation']}")

    print("\nBest solution (model_selection='best'):")
    best1 = model1.sympy()
    print(f"  {best1}")

    print("\nKONFIGURACJA 2")
    print("  binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos',
    ↪ 'exp', 'sin', 'log']")
```

```

model2 = PySRRegressor(
    binary_operators=["+", "*", "-", "^"],
    unary_operators=["cos", "exp", "sin", "log"],
    maxsize=30,
    constraints={"^": (-1, 1)},
    **default_pysr_params,
)
with warnings.catch_warnings():
    warnings.filterwarnings(
        "ignore",
        message=r"Note: it looks like you are running in Jupyter.*",
        category=UserWarning,
    )
    model2.fit(X, y)

print("\nTop 3 rozwiązania (maksymalny score):")
eq2 = model2.equations_
top3_eq2 = eq2.nlargest(3, 'score')[['complexity', 'loss', 'score',
↪ 'equation']]
for idx, row in top3_eq2.iterrows():
    print(f"    [{row['complexity']}] Score: {row['score']:.6f}, Loss:
↪ {row['loss']:.6f}")
    print(f"    Równanie: {row['equation']}")

print("\nBest solution (model_selection='best'):")
best2 = model2.sympy()
print(f"    {best2}")

# Konfiguracja 3
print("\nKONFIGURACJA 3")
print("    binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp',
↪ 'sin']")

model3 = PySRRegressor(
    binary_operators=["+", "*", "-", "^"],
    unary_operators=["exp", "sin"],
    maxsize=15,
    constraints={"^": (-1, 1)},
    **default_pysr_params,
)
with warnings.catch_warnings():
    warnings.filterwarnings(

```

```

        "ignore",
        message=r"Note: it looks like you are running in Jupyter.*",
        category=UserWarning,
    )
    model3.fit(X, y)

    print("\nTop 3 rozwiązania (maksymalny score):")
    eq3 = model3.equations_
    top3_eq3 = eq3.nlargest(3, 'score')[['complexity', 'loss', 'score',
    ↪ 'equation']]
    for idx, row in top3_eq3.iterrows():
        print(f"    [{row['complexity']}] Score: {row['score']:.6f}, Loss:
        ↪ {row['loss']:.6f}")
        print(f"    Równanie: {row['equation']}")

    print("\nBest solution (model_selection='best'):")
    best3 = model3.sympy()
    print(f"    {best3}")

    return model1, model2, model3

```

```

print("Próby na próbce bez szumu")
model1, model2, model3 = run_all_setups(X, y)

```

Próby na próbce bez szumu

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 4.491950, Loss: 8.701544

Równanie: $x_5 * (x_5 * -1.0570904)$

[7] Score: 0.648067, Loss: 2.380635

Równanie: $(x_5 * (x_5 * -1.00369)) + -3.1152098$

[15] Score: 0.417232, Loss: 1.465831

Równanie: $((x_5 * (x_5 * -1.0036206)) + -3.0735738) + (\cos(x_1 * 1.9995271) * \sin(x_0))$

Best solution (model_selection='best'):

$x_5 * x_5 * (-1.0036206) + \sin(x_0) * \cos(x_1 * 1.9995271) - 3.0735738$

KONFIGURACJA 2

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',  
↪ 'sin', 'log']
```

Top 3 rozwiązania (maksymalny score):

```
[14] Score: 12.900695, Loss: 0.000000  
Równanie: ((sin((x1 + x1) + x0) + -1.3636369) * 2.1999998) - (x5 * x5)  
[5] Score: 5.783166, Loss: 2.391159  
Równanie: -3.1902225 - (x5 * x5)  
[12] Score: 1.164398, Loss: 0.711419  
Równanie: (sin(x0 + (x1 + x1)) + -3.103784) - (x5 * x5)
```

Best solution (model_selection='best'):

```
-x5*x5 + (sin(x0 + x1 + x1) - 1.3636369)*2.1999998
```

KONFIGURACJA 3

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']
```

Top 3 rozwiązania (maksymalny score):

```
[14] Score: 12.798015, Loss: 0.000000  
Równanie: (sin((x1 + x1) + x0) * 2.2000008) - ((x5 * x5) + 3.0000002)  
[5] Score: 5.783166, Loss: 2.391159  
Równanie: -3.1901615 - (x5 * x5)  
[12] Score: 1.140163, Loss: 0.711419  
Równanie: sin((x1 + x1) + x0) - ((x5 * x5) + 3.1037815)
```

Best solution (model_selection='best'):

```
-(x5*x5 + 3.0000002) + sin(x0 + x1 + x1)*2.2000008
```

Szumy

Przygotowano próbki wartości y z dodanym szumem o $\text{std} = 0.5$. Następnie powtórzono poprzedni eksperyment.

```
np.random.seed(0)  
X_noisy = 5 * np.random.randn(200, 6)  
noise = np.random.normal(0, 0.5, 200)  
y_noisy = 2.2 * np.sin(X_noisy[:, 0] + 2 * X_noisy[:, 1]) - X_noisy[:, 5] **  
↪ 2 - 3 + noise
```

```
print("Próby na próbce z szumem o std=0.5")  
model1_n, model2_n, model3_n = run_all_setups(X_noisy, y_noisy)
```

Próby na próbce z szumem o std=0.5

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 4.456694, Loss: 9.009994

Równanie: $x_5 * (x_5 * -1.056797)$

[7] Score: 0.611257, Loss: 2.653342

Równanie: $(x_5 * (x_5 * -1.003245)) + -3.1240578$

[13] Score: 0.040769, Loss: 2.481600

Równanie: $(\sin(\exp(x_5 * 0.39229316)) + -3.6443965) + ((x_5 * -0.99495035) * x_5)$

Best solution (model_selection='best'):

$x_5 * x_5 * (-1.003245) - 3.1240578$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp', 'sin', 'log']

Top 3 rozwiązania (maksymalny score):

[5] Score: 5.675519, Loss: 2.661484

Równanie: $-3.1900318 - (x_5 * x_5)$

[14] Score: 0.972046, Loss: 0.961393

Równanie: $((x_5 * (x_5 * -1.001537)) + -3.073882) + \sin((x_1 * 2.0077116) + x_0)$

[16] Score: 0.709620, Loss: 0.232559

Równanie: $((x_5 * -0.9994969) * x_5) + -3.0099137 + (\sin(x_0 + (x_1 * 2.0032969)) * 2.2168362)$

Best solution (model_selection='best'):

$x_5 * (-0.9994969) * x_5 + \sin(x_0 + x_1 * 2.0032969) * 2.2168362 - 3.0099137$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 5.675520, Loss: 2.661484

Równanie: $-3.1904702 - (x_5 * x_5)$

[12] Score: 0.933613, Loss: 0.965595

Równanie: $(-3.0803666 - (x_5 * x_5)) + \sin(x_1 + (x_1 + x_0))$

[11] Score: 0.047316, Loss: 2.456167

Równanie: $-3.2157106 - (\sin(\exp(x_1) * -6.4725003) + (x_5 * x_5))$

Best solution (model_selection='best'):
 $-x_5 \cdot x_5 + \sin(x_0 + x_1 + x_1) - 3.0803666$

```

results = []
for name, model, noisy in [
    ("Config 1", model1, False),
    ("Config 2", model2, False),
    ("Config 3", model3, False),
    ("Config 1", model1_n, True),
    ("Config 2", model2_n, True),
    ("Config 3", model3_n, True),
]:
    y_pred = model.predict(X)
    r2 = r2_score(y, y_pred)
    best_eq = model.sympy()
    equations_df = model.equations_
    best_matches = equations_df[equations_df['equation'] == str(best_eq)]
    if best_matches.empty:
        best_row = equations_df.sort_values(['score', 'loss'],
        ↪ ascending=[False, True]).iloc[0]
    else:
        best_row = best_matches.iloc[0]

    results.append({
        "Konfiguracja": name,
        "Szum": "Tak" if noisy else "Nie",
        "Best Score": f"{best_row['score']:.6f}",
        "Best Loss": f"{best_row['loss']:.6f}",
        "R² (treningowy)": f"{r2:.6f}",
        "Złożoność": int(best_row['complexity']),
    })

results_df = pd.DataFrame(results)
print("\n" + results_df.to_string(index=False))

```

Konfiguracja	Szum	Best Score	Best Loss	R ² (treningowy)	Złożoność
Config 1	Nie	4.491950	8.701544	0.998123	5
Config 2	Nie	12.900695	0.000000	1.000000	14
Config 3	Nie	12.798015	0.000000	1.000000	14
Config 1	Tak	4.456694	9.009994	0.996951	5

Config 2	Tak	5.675519	2.661484	0.999999	5
Config 3	Tak	5.675520	2.661484	0.999088	5

Wizualizacja dokładności predykcji

Najpierw na całym zakresie przyjmowanych wartości, a następnie na zakresie wartości uciętym do (-25, 0) żeby dało się coś zobaczyć.

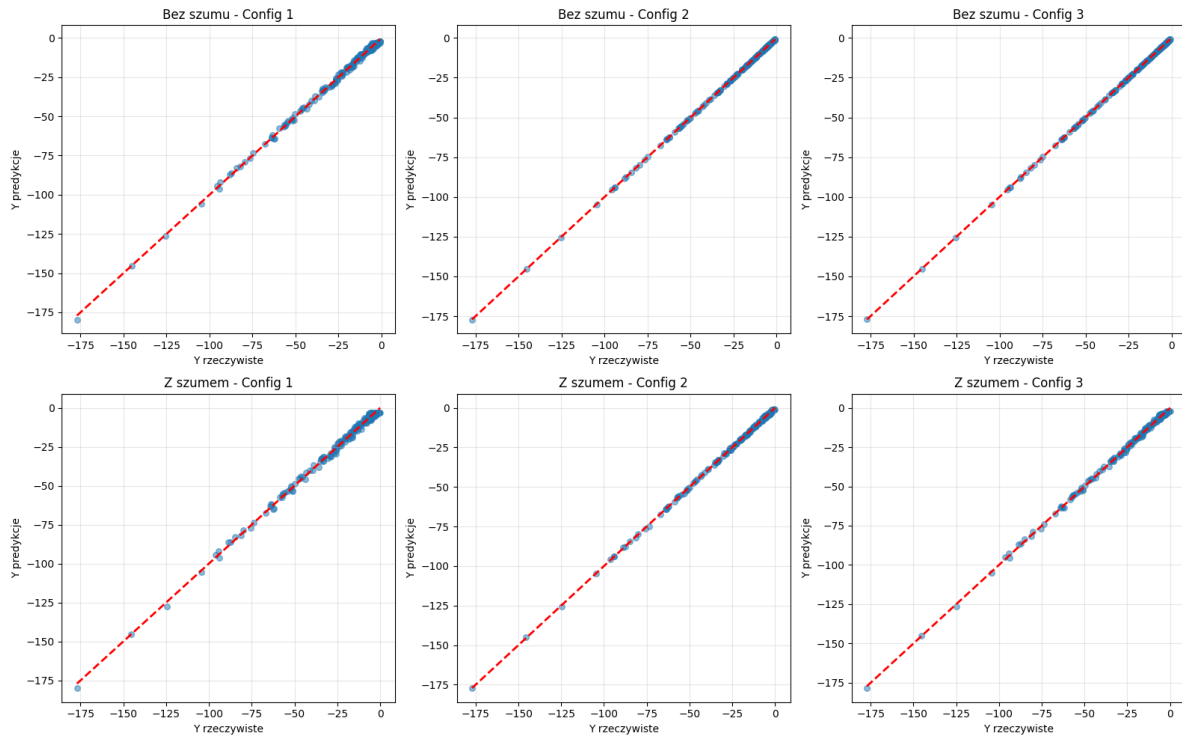
```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

models_no_noise = [model1, model2, model3]
models_noise = [model1_n, model2_n, model3_n]
titles = ["Config 1", "Config 2", "Config 3"]

for idx, (model, ax, title) in enumerate(zip(models_no_noise, axes[0],
↪ titles)):
    y_pred = model.predict(X)
    ax.scatter(y, y_pred, alpha=0.5, s=30)
    ax.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
    ax.set_xlabel("Y rzeczywiste")
    ax.set_ylabel("Y predykcje")
    ax.set_title(f"Bez szumu - {title}")
    ax.grid(True, alpha=0.3)

for idx, (model, ax, title) in enumerate(zip(models_noise, axes[1], titles)):
    y_pred = model.predict(X_noisy)
    ax.scatter(y_noisy, y_pred, alpha=0.5, s=30)
    ax.plot([y_noisy.min(), y_noisy.max()], [y_noisy.min(), y_noisy.max()],
↪ 'r--', lw=2)
    ax.set_xlabel("Y rzeczywiste")
    ax.set_ylabel("Y predykcje")
    ax.set_title(f"Z szumem - {title}")
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

models_no_noise = [model1, model2, model3]
models_noise = [model1_n, model2_n, model3_n]
titles = ["Config 1", "Config 2", "Config 3"]

for idx, (model, ax, title) in enumerate(zip(models_no_noise, axes[0],
↪ titles)):
    y_pred = model.predict(X)
    ax.scatter(y, y_pred, alpha=0.5, s=30)
    ax.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
    ax.set_xlabel("Y rzeczywiste")
    ax.set_ylabel("Y predykcje")
    ax.set_title(f"Bez szumu - {title}")
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-25, 0)
    ax.set_xlim(-25, 0)

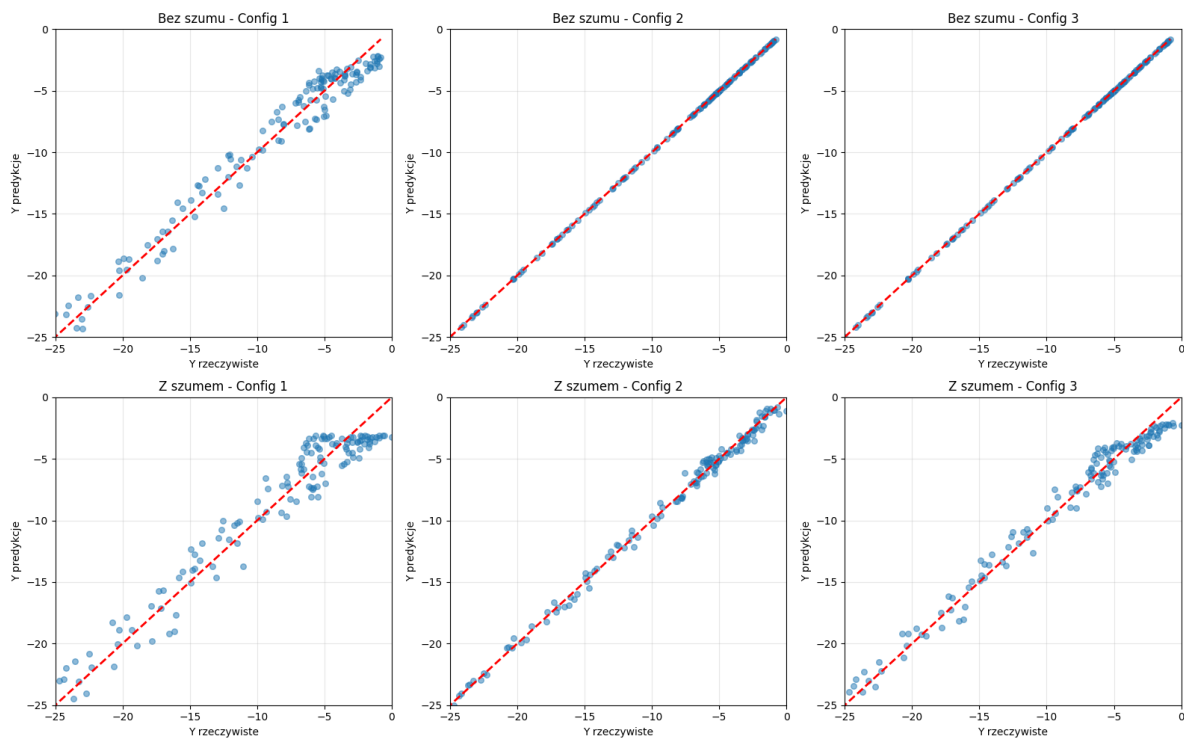
for idx, (model, ax, title) in enumerate(zip(models_noise, axes[1], titles)):
    y_pred = model.predict(X_noisy)
    ax.scatter(y_noisy, y_pred, alpha=0.5, s=30)
```

```

ax.plot([y_noisy.min(), y_noisy.max()], [y_noisy.min(), y_noisy.max()],
↪ 'r--', lw=2)
ax.set_xlabel("Y rzeczywiste")
ax.set_ylabel("Y predykcje")
ax.set_title(f"Z szumem - {title}")
ax.grid(True, alpha=0.3)
ax.set_ylim(-25, 0)
ax.set_xlim(-25, 0)

plt.tight_layout()
plt.show()

```



Na 4.0

Zmieniono zakres wartości w próbce na między -15 i 15.

Następnie przygotowano 4 próbki wartości y z różnymi poziomami szumu (w tym większymi niż poprzednio).

Są to szумы o wartościach std: - 0 - brak szumu - 0.5 - 2.0 - 5.0

```

np.random.seed(0)
X15 = 15 * np.random.randn(200, 6)
y15_no_noise = 2.2 * np.sin(X15[:, 0] + 2 * X15[:, 1]) - X15[:, 5] ** 2 - 3

noise0_5 = np.random.normal(0, 0.5, 200)
y_noisy0_5 = y15_no_noise + noise0_5

noise2 = np.random.normal(0, 2, 200)
y_noisy2 = y15_no_noise + noise2

noise5 = np.random.normal(0, 5, 200)
y_noisy5 = y15_no_noise + noise5

print("Próby na próbce bez szumu")
model15_1, model15_2, model15_3 = run_all_setups(X15, y15_no_noise)

```

Próby na próbce bez szumu

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[16] Score: 21.154972, Loss: 0.000000

Równanie: ((sin(x1 + (x1 + x0)) * 2.1999924) + -3.000006) + (x5 * (x5 *
↪ -1.0))

[5] Score: 4.416381, Loss: 9.107293

Równanie: x5 * (x5 * -1.0061768)

[7] Score: 0.610404, Loss: 2.686576

Równanie: (x5 * (x5 * -1.000199)) + -3.1396942

Best solution (model_selection='best'):

x5*x5*(-1.0) + sin(x0 + x1 + x1)*2.1999924 - 3.000006

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log']

Top 3 rozwiązania (maksymalny score):

[5] Score: 5.024888, Loss: 2.689034

Równanie: -3.1759644 - (x5 * x5)

[10] Score: 0.123989, Loss: 2.347394

Równanie: sin(2.4891489 ^ x3) - ((x5 * x5) + 3.1848605)

[21] Score: 0.031322, Loss: 2.120697
Równanie: $(\exp(\sin((2.4891489 \wedge x3) - \sin((\exp(x5) * -0.40707305) * x1))) -$
↪ $((x5 * x5) + 5.596971)) + 1.2008708$

Best solution (model_selection='best'):
-x5*x5 - 3.1759644

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 5.024888, Loss: 2.689033
Równanie: $-3.1759145 - (x5 * x5)$
[12] Score: 0.106850, Loss: 2.387948
Równanie: $(x5 * (x5 * -0.9996179)) + (\sin(0.305276 \wedge x3) + -3.3190343)$
[7] Score: 0.005274, Loss: 2.660820
Równanie: $((0.012434402 - x5) * x5) + -3.1661212$

Best solution (model_selection='best'):
-x5*x5 - 3.1759145

```
print("Próby na próbce z szumem o std=0.5")
model15_1_n0_5, model15_2_n0_5, model15_3_n0_5 = run_all_setups(X15,
↪ y_noisy0_5)
```

Próby na próbce z szumem o std=0.5

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 4.392777, Loss: 9.546580
Równanie: $x5 * (x5 * -1.0061408)$
[14] Score: 0.942560, Loss: 1.125384
Równanie: $\sin((x0 + x1) + x1) + ((x5 * (x5 * -1.0000589)) + -3.0850358)$
[16] Score: 0.792219, Loss: 0.230775
Równanie: $((x5 * -0.99994403) * x5) + ((\sin((x1 + x1) + x0) * 2.2700334) +$
↪ $-3.004169)$

Best solution (model_selection='best'):
 $x5*(-0.99994403)*x5 + \sin(x0 + x1 + x1)*2.2700334 - 3.004169$

KONFIGURACJA 2

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',  
↪ 'sin', 'log']
```

Top 3 rozwiązania (maksymalny score):

```
[5] Score: 4.955148, Loss: 3.091219  
Równanie: -3.1757202 - (x5 * x5)  
[14] Score: 1.494599, Loss: 0.230970  
Równanie: ((sin((x1 + x1) + x0) * 2.268922) - (x5 * x5)) + -2.99417  
[12] Score: 0.951945, Loss: 1.125603  
Równanie: (sin((x0 + x1) + x1) - (x5 * x5)) + -3.0957336
```

Best solution (model_selection='best'):

```
-x5*x5 + sin(x0 + x1 + x1)*2.268922 - 2.99417
```

KONFIGURACJA 3

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']
```

Top 3 rozwiązania (maksymalny score):

```
[5] Score: 4.955148, Loss: 3.091219  
Równanie: -3.1757202 - (x5 * x5)  
[12] Score: 0.949442, Loss: 1.125603  
Równanie: (sin((x0 + x1) + x1) - (x5 * x5)) - 3.0957336  
[13] Score: 0.089189, Loss: 1.029559  
Równanie: (exp(sin((x1 + x1) + x0)) + -4.3834476) - (x5 * x5)
```

Best solution (model_selection='best'):

```
-x5*x5 + sin(x0 + x1 + x1) - 1*3.0957336
```

```
print("Próby na próbce z szumem o std=2.0")  
model15_1_n2, model15_2_n2, model15_3_n2 = run_all_setups(X15, y_noisy2)
```

Próby na próbce z szumem o std=2.0

KONFIGURACJA 1

```
binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']
```

Top 3 rozwiązania (maksymalny score):

```
[5] Score: 4.254380, Loss: 12.585590  
Równanie: (x5 * x5) * -1.0059495  
[7] Score: 0.373152, Loss: 5.967025  
Równanie: (x5 * (x5 * -0.9998776)) + -3.1878488  
[12] Score: 0.034817, Loss: 5.670306  
Równanie: cos(x0 * 0.5376156) + ((x5 * (x5 * -0.99998534)) + -3.1867516)
```

Best solution (model_selection='best'):

$x_5 * x_5 * (-0.9998776) - 3.1878488$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log']

Top 3 rozwiązania (maksymalny score):

[5] Score: 4.625941, Loss: 5.967993

Równanie: $-3.1661682 - (x_5 * x_5)$

[12] Score: 0.216810, Loss: 4.420483

Równanie: $(\sin(x_0 + (x_1 + x_1)) + -3.085013) - (x_5 * x_5)$

[10] Score: 0.062920, Loss: 5.514429

Równanie: $\sin(2.5137296 \wedge x_0) + (-3.085013 - (x_5 * x_5))$

Best solution (model_selection='best'):

$-x_5 * x_5 + \sin(x_0 + x_1 + x_1) - 3.085013$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 4.625941, Loss: 5.967994

Równanie: $-3.1652036 - (x_5 * x_5)$

[10] Score: 0.042848, Loss: 5.639453

Równanie: $(\sin(x_5 * -5.610089e7) + -3.106418) - (x_5 * x_5)$

[11] Score: 0.013364, Loss: 5.564588

Równanie: $\sin(\exp(x_3 - 4.926522)) - ((x_5 * x_5) + 3.221912)$

Best solution (model_selection='best'):

$-x_5 * x_5 - 3.1652036$

```
print("Próby na próbce z szumem o std=5.0")
model15_1_n5, model15_2_n5, model15_3_n5 = run_all_setups(X15, y_noisy5)
```

Próby na próbce z szumem o std=5.0

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin']

Top 3 rozwiązania (maksymalny score):

[5] Score: 3.761354, Loss: 33.759087

```

Równanie: (x5 * x5) * -1.0060574
[7] Score: 0.108442, Loss: 27.176844
Równanie: ((x5 * -0.9999973) * x5) + -3.1793935
[12] Score: 0.031694, Loss: 25.855497
Równanie: sin(x4 * 14.165813) + (((x5 * x5) * -1.0002053) + -3.0774164)

Best solution (model_selection='best'):
x5*x5*(-1.0060574)

KONFIGURACJA 2
binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log']

Top 3 rozwiązania (maksymalny score):
[5] Score: 3.868211, Loss: 27.176857
Równanie: -3.1786587 - (x5 * x5)
[20] Score: 0.031140, Loss: 23.609978
Równanie: (-0.18443379 - (x5 * x5)) - exp(cos((x0 * 20.243883) - ((x5 *
↪ -0.073050365) - sin(x1)))) - -0.82581425)
[10] Score: 0.029906, Loss: 25.926186
Równanie: -3.140283 - ((x5 * x5) + cos(x0 * 20.254076))

Best solution (model_selection='best'):
-x5*x5 - 3.1786587

KONFIGURACJA 3
binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin']

Top 3 rozwiązania (maksymalny score):
[5] Score: 3.868211, Loss: 27.176857
Równanie: -3.1784337 - (x5 * x5)
[10] Score: 0.035873, Loss: 25.858871
Równanie: -3.1080012 - (sin(x4 * -14.166061) + (x5 * x5))
[15] Score: 0.016166, Loss: 25.136702
Równanie: -3.1354666 - ((sin(x4 * -14.166304) + sin(x4 * 2.1777723)) + (x5
↪ * x5))

Best solution (model_selection='best'):
-x5*x5 - 3.1784337

```

```

results = []
for name, model, noisy_level, target_y in [

```

```

("Config 1", model15_1, "0", y15_no_noise),
("Config 2", model15_2, "0", y15_no_noise),
("Config 3", model15_3, "0", y15_no_noise),
("Config 1", model15_1_n0_5, "0.5", y_noisy0_5),
("Config 2", model15_2_n0_5, "0.5", y_noisy0_5),
("Config 3", model15_3_n0_5, "0.5", y_noisy0_5),
("Config 1", model15_1_n2, "2.0", y_noisy2),
("Config 2", model15_2_n2, "2.0", y_noisy2),
("Config 3", model15_3_n2, "2.0", y_noisy2),
("Config 1", model15_1_n5, "5.0", y_noisy5),
("Config 2", model15_2_n5, "5.0", y_noisy5),
("Config 3", model15_3_n5, "5.0", y_noisy5),
]:
    y_pred = model.predict(X15)
    r2 = r2_score(target_y, y_pred)
    best_eq = model.sympy()
    equations_df = model.equations_
    best_matches = equations_df[equations_df['equation'] == str(best_eq)]
    if best_matches.empty:
        best_row = equations_df.sort_values(['score', 'loss'],
        ↪ ascending=[False, True]).iloc[0]
    else:
        best_row = best_matches.iloc[0]

    results.append({
        "Konfiguracja": name,
        "Szum": noisy_level,
        "Best Score": f"{best_row['score']:.6f}",
        "Best Loss": f"{best_row['loss']:.6f}",
        "R² (treningowy)": f"{r2:.6f}",
        "Złożoność": int(best_row['complexity']),
    })

results_df = pd.DataFrame(results)
print("\n" + results_df.to_string(index=False))

```

Konfiguracja	Szum	Best Score	Best Loss	R ² (treningowy)	Złożoność
Config 1	0	21.154972	0.000000	1.000000	16
Config 2	0	5.024888	2.689034	0.999957	5
Config 3	0	5.024888	2.689033	0.999957	5
Config 1	0.5	4.392777	9.546580	0.999996	5

Config 2	0.5	4.955148	3.091219	0.999996	5
Config 3	0.5	4.955148	3.091219	0.999982	5
Config 1	2.0	4.254380	12.585590	0.999905	5
Config 2	2.0	4.625941	5.967993	0.999929	5
Config 3	2.0	4.625941	5.967994	0.999905	5
Config 1	5.0	3.761354	33.759087	0.999461	5
Config 2	5.0	3.868211	27.176857	0.999566	5
Config 3	5.0	3.868211	27.176857	0.999566	5

```
import seaborn as sns

heat_df = results_df.copy()

for col in ["Best Score", "Best Loss", "R2 (treningowy)", "Złożoność"]:
    heat_df[col] = pd.to_numeric(heat_df[col], errors="coerce")

noise_order = ["0", "0.5", "2.0", "5.0"]
config_order = ["Config 1", "Config 2", "Config 3"]

heat_df["Szum"] = pd.Categorical(heat_df["Szum"], categories=noise_order,
    ↪ ordered=True)
heat_df["Konfiguracja"] = pd.Categorical(heat_df["Konfiguracja"],
    ↪ categories=config_order, ordered=True)
heat_df = heat_df.sort_values(["Szum", "Konfiguracja"])

metrics = ["Best Score", "Best Loss", "R2 (treningowy)"]
cmaps = ["YlGnBu", "YlOrRd", "PuBuGn", "magma"]

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes = axes.flatten()

for ax, metric, cmap in zip(axes, metrics, cmaps):
    pivot = heat_df.pivot(index="Szum", columns="Konfiguracja",
    ↪ values=metric).reindex(
        index=noise_order, columns=config_order
    )
    fmt = ".0f" if metric == "Złożoność" else ".4f"

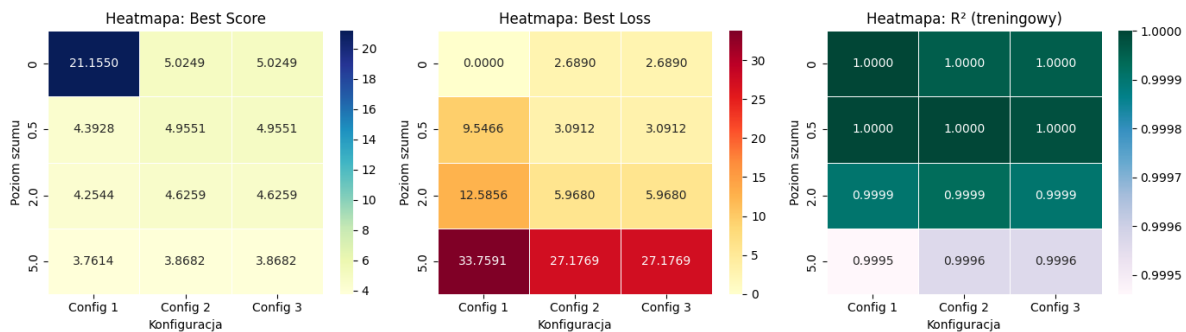
    sns.heatmap(
        pivot,
        annot=True,
        fmt=fmt,
```

```

        cmap=cmap,
        cbar=True,
        linewidths=0.5,
        linecolor="white",
        ax=ax,
    )
    ax.set_title(f"Heatmapa: {metric}")
    ax.set_xlabel("Konfiguracja")
    ax.set_ylabel("Poziom szum")

plt.tight_layout()
plt.show()

```



Wizualizacja dokładności predykcji

Przyjmowane wartości obcięte do między (-200, -100) oraz (-100, 0) ponieważ na całym zakresie wartości na wykresach nie widac różnicy (skala błędu jest nieporównywalnie mniejsza od skali osiągniętych wartości).

```

fig, axes = plt.subplots(4, 3, figsize=(16, 20))
titles = ["Config 1", "Config 2", "Config 3"]

plot_rows = [
    ("Bez szumu", models_no_noise, y15_no_noise, X15),
    ("Szum 0.5", [model15_1_n0_5, model15_2_n0_5, model15_3_n0_5],
    ↪ y_noisy0_5, X15),
    ("Szum 2.0", [model15_1_n2, model15_2_n2, model15_3_n2], y_noisy2, X15),
    ("Szum 5.0", [model15_1_n5, model15_2_n5, model15_3_n5], y_noisy5, X15),
]

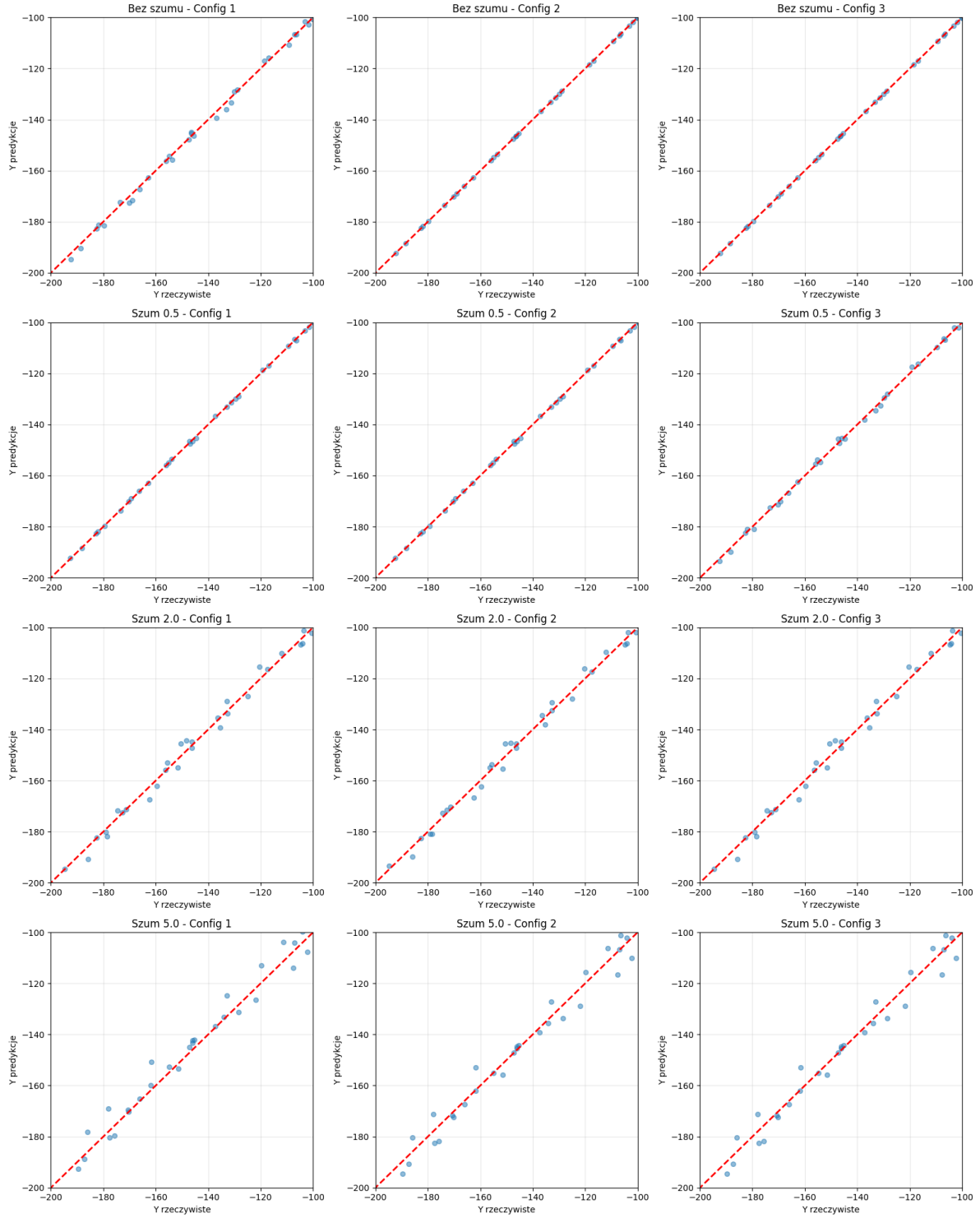
```

```

for row_idx, (row_title, row_models, target_y, X_data) in
    ↪ enumerate(plot_rows):
    for col_idx, (model, ax, title) in enumerate(zip(row_models,
        ↪ axes[row_idx], titles)):
        y_pred = model.predict(X_data)
        ax.scatter(target_y, y_pred, alpha=0.5, s=30)
        ax.plot([target_y.min(), target_y.max()], [target_y.min(),
    ↪ target_y.max()], 'r--', lw=2)
        ax.set_xlabel("Y rzeczywiste")
        ax.set_ylabel("Y predykcje")
        ax.set_title(f"{row_title} - {title}")
        ax.grid(True, alpha=0.3)
        ax.set_ylim(-200, -100)
        ax.set_xlim(-200, -100)

plt.tight_layout()
plt.show()

```



```

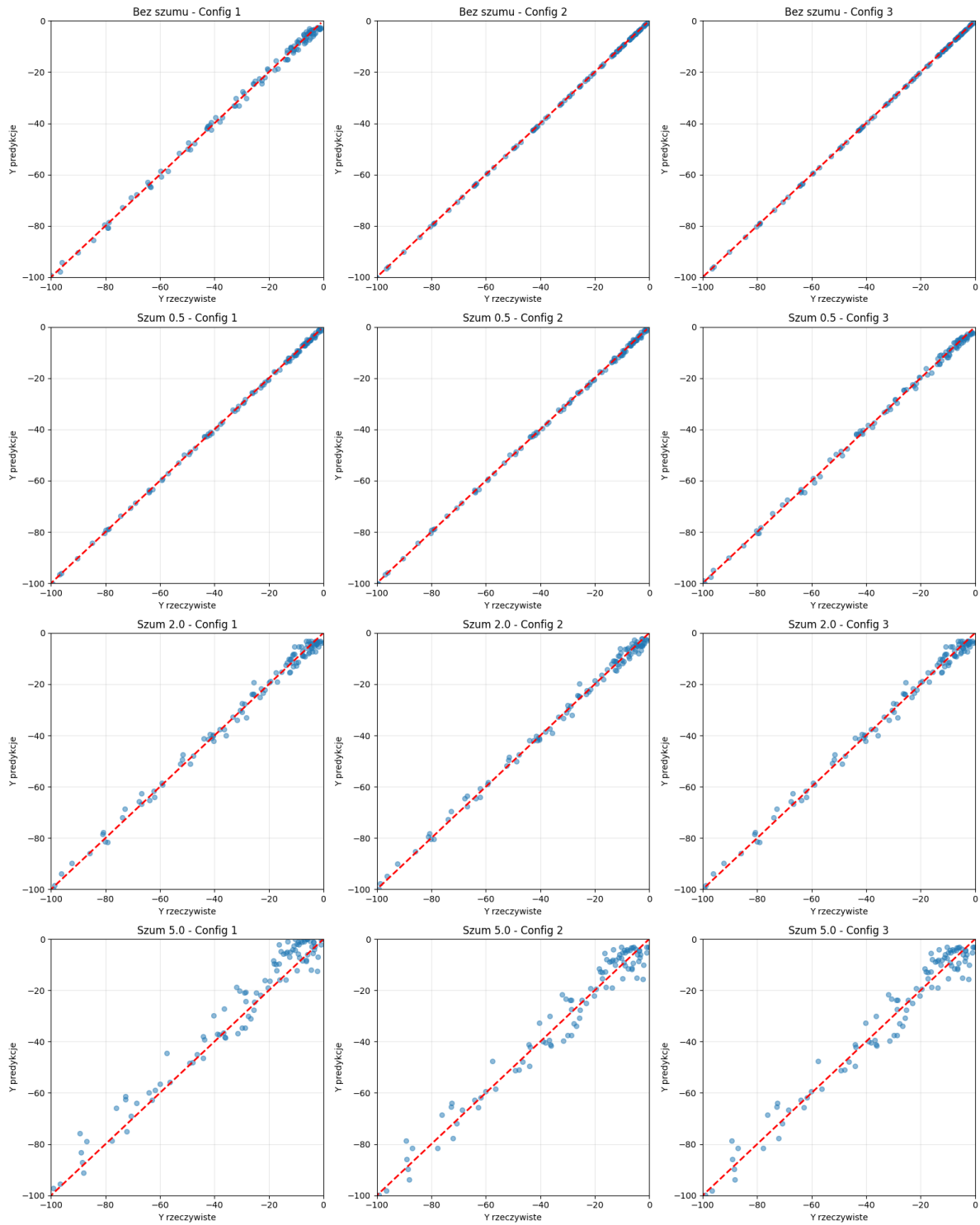
fig, axes = plt.subplots(4, 3, figsize=(16, 20))
titles = ["Config 1", "Config 2", "Config 3"]

plot_rows = [
    ("Bez szumu", models_no_noise, y15_no_noise, X15),
    ("Szum 0.5", [model15_1_n0_5, model15_2_n0_5, model15_3_n0_5],
    ↪ y_noisy0_5, X15),
    ("Szum 2.0", [model15_1_n2, model15_2_n2, model15_3_n2], y_noisy2, X15),
    ("Szum 5.0", [model15_1_n5, model15_2_n5, model15_3_n5], y_noisy5, X15),
]

for row_idx, (row_title, row_models, target_y, X_data) in
    ↪ enumerate(plot_rows):
    for col_idx, (model, ax, title) in enumerate(zip(row_models,
    ↪ axes[row_idx], titles)):
        y_pred = model.predict(X_data)
        ax.scatter(target_y, y_pred, alpha=0.5, s=30)
        ax.plot([target_y.min(), target_y.max()], [target_y.min(),
    ↪ target_y.max()], 'r--', lw=2)
        ax.set_xlabel("Y rzeczywiste")
        ax.set_ylabel("Y predykcje")
        ax.set_title(f"{row_title} - {title}")
        ax.grid(True, alpha=0.3)
        ax.set_ylim(-100, -0)
        ax.set_xlim(-100, -0)

plt.tight_layout()
plt.show()

```



Na 5.0

Funkcja: $f(x) = 2.2 \sin(x_0 + 2x_1) - x_5^2 - p(\lfloor x_0 \rfloor)$ gdzie $p(i)$ oznacza i -tą liczbę pierwszą, z dziedziną jest $\mathbb{R}^6$

```
import numpy as np
import sympy
from pysr import PySRRegressor, jl
jl.seval(
    """
import Pkg
Pkg.add("Primes")
"""
)
jl.seval("using Primes: prime")

jl.seval(
    """
function p(i::T) where T
    if 0.5 < i < 1000
        return T(prime(floor{Int, i}))
    else
        return T{NaN}
    end
end
""")

primes = {i: jl.p(float(i)) for i in range(1, 1000)}
import sympy

class sympy_p(sympy.Function):
    pass
```

```
Resolving package versions...
No Changes to `~/julia/environments/pyjuliapkg/Project.toml`
No Changes to `~/julia/environments/pyjuliapkg/Manifest.toml`
```

```
primes
```

```
{1: 2.0,
 2: 3.0,
```

3: 5.0,
4: 7.0,
5: 11.0,
6: 13.0,
7: 17.0,
8: 19.0,
9: 23.0,
10: 29.0,
11: 31.0,
12: 37.0,
13: 41.0,
14: 43.0,
15: 47.0,
16: 53.0,
17: 59.0,
18: 61.0,
19: 67.0,
20: 71.0,
21: 73.0,
22: 79.0,
23: 83.0,
24: 89.0,
25: 97.0,
26: 101.0,
27: 103.0,
28: 107.0,
29: 109.0,
30: 113.0,
31: 127.0,
32: 131.0,
33: 137.0,
34: 139.0,
35: 149.0,
36: 151.0,
37: 157.0,
38: 163.0,
39: 167.0,
40: 173.0,
41: 179.0,
42: 181.0,
43: 191.0,
44: 193.0,
45: 197.0,

46: 199.0,
47: 211.0,
48: 223.0,
49: 227.0,
50: 229.0,
51: 233.0,
52: 239.0,
53: 241.0,
54: 251.0,
55: 257.0,
56: 263.0,
57: 269.0,
58: 271.0,
59: 277.0,
60: 281.0,
61: 283.0,
62: 293.0,
63: 307.0,
64: 311.0,
65: 313.0,
66: 317.0,
67: 331.0,
68: 337.0,
69: 347.0,
70: 349.0,
71: 353.0,
72: 359.0,
73: 367.0,
74: 373.0,
75: 379.0,
76: 383.0,
77: 389.0,
78: 397.0,
79: 401.0,
80: 409.0,
81: 419.0,
82: 421.0,
83: 431.0,
84: 433.0,
85: 439.0,
86: 443.0,
87: 449.0,
88: 457.0,

89: 461.0,
90: 463.0,
91: 467.0,
92: 479.0,
93: 487.0,
94: 491.0,
95: 499.0,
96: 503.0,
97: 509.0,
98: 521.0,
99: 523.0,
100: 541.0,
101: 547.0,
102: 557.0,
103: 563.0,
104: 569.0,
105: 571.0,
106: 577.0,
107: 587.0,
108: 593.0,
109: 599.0,
110: 601.0,
111: 607.0,
112: 613.0,
113: 617.0,
114: 619.0,
115: 631.0,
116: 641.0,
117: 643.0,
118: 647.0,
119: 653.0,
120: 659.0,
121: 661.0,
122: 673.0,
123: 677.0,
124: 683.0,
125: 691.0,
126: 701.0,
127: 709.0,
128: 719.0,
129: 727.0,
130: 733.0,
131: 739.0,

132: 743.0,
133: 751.0,
134: 757.0,
135: 761.0,
136: 769.0,
137: 773.0,
138: 787.0,
139: 797.0,
140: 809.0,
141: 811.0,
142: 821.0,
143: 823.0,
144: 827.0,
145: 829.0,
146: 839.0,
147: 853.0,
148: 857.0,
149: 859.0,
150: 863.0,
151: 877.0,
152: 881.0,
153: 883.0,
154: 887.0,
155: 907.0,
156: 911.0,
157: 919.0,
158: 929.0,
159: 937.0,
160: 941.0,
161: 947.0,
162: 953.0,
163: 967.0,
164: 971.0,
165: 977.0,
166: 983.0,
167: 991.0,
168: 997.0,
169: 1009.0,
170: 1013.0,
171: 1019.0,
172: 1021.0,
173: 1031.0,
174: 1033.0,

175: 1039.0,
176: 1049.0,
177: 1051.0,
178: 1061.0,
179: 1063.0,
180: 1069.0,
181: 1087.0,
182: 1091.0,
183: 1093.0,
184: 1097.0,
185: 1103.0,
186: 1109.0,
187: 1117.0,
188: 1123.0,
189: 1129.0,
190: 1151.0,
191: 1153.0,
192: 1163.0,
193: 1171.0,
194: 1181.0,
195: 1187.0,
196: 1193.0,
197: 1201.0,
198: 1213.0,
199: 1217.0,
200: 1223.0,
201: 1229.0,
202: 1231.0,
203: 1237.0,
204: 1249.0,
205: 1259.0,
206: 1277.0,
207: 1279.0,
208: 1283.0,
209: 1289.0,
210: 1291.0,
211: 1297.0,
212: 1301.0,
213: 1303.0,
214: 1307.0,
215: 1319.0,
216: 1321.0,
217: 1327.0,

218: 1361.0,
219: 1367.0,
220: 1373.0,
221: 1381.0,
222: 1399.0,
223: 1409.0,
224: 1423.0,
225: 1427.0,
226: 1429.0,
227: 1433.0,
228: 1439.0,
229: 1447.0,
230: 1451.0,
231: 1453.0,
232: 1459.0,
233: 1471.0,
234: 1481.0,
235: 1483.0,
236: 1487.0,
237: 1489.0,
238: 1493.0,
239: 1499.0,
240: 1511.0,
241: 1523.0,
242: 1531.0,
243: 1543.0,
244: 1549.0,
245: 1553.0,
246: 1559.0,
247: 1567.0,
248: 1571.0,
249: 1579.0,
250: 1583.0,
251: 1597.0,
252: 1601.0,
253: 1607.0,
254: 1609.0,
255: 1613.0,
256: 1619.0,
257: 1621.0,
258: 1627.0,
259: 1637.0,
260: 1657.0,

261: 1663.0,
262: 1667.0,
263: 1669.0,
264: 1693.0,
265: 1697.0,
266: 1699.0,
267: 1709.0,
268: 1721.0,
269: 1723.0,
270: 1733.0,
271: 1741.0,
272: 1747.0,
273: 1753.0,
274: 1759.0,
275: 1777.0,
276: 1783.0,
277: 1787.0,
278: 1789.0,
279: 1801.0,
280: 1811.0,
281: 1823.0,
282: 1831.0,
283: 1847.0,
284: 1861.0,
285: 1867.0,
286: 1871.0,
287: 1873.0,
288: 1877.0,
289: 1879.0,
290: 1889.0,
291: 1901.0,
292: 1907.0,
293: 1913.0,
294: 1931.0,
295: 1933.0,
296: 1949.0,
297: 1951.0,
298: 1973.0,
299: 1979.0,
300: 1987.0,
301: 1993.0,
302: 1997.0,
303: 1999.0,

304: 2003.0,
305: 2011.0,
306: 2017.0,
307: 2027.0,
308: 2029.0,
309: 2039.0,
310: 2053.0,
311: 2063.0,
312: 2069.0,
313: 2081.0,
314: 2083.0,
315: 2087.0,
316: 2089.0,
317: 2099.0,
318: 2111.0,
319: 2113.0,
320: 2129.0,
321: 2131.0,
322: 2137.0,
323: 2141.0,
324: 2143.0,
325: 2153.0,
326: 2161.0,
327: 2179.0,
328: 2203.0,
329: 2207.0,
330: 2213.0,
331: 2221.0,
332: 2237.0,
333: 2239.0,
334: 2243.0,
335: 2251.0,
336: 2267.0,
337: 2269.0,
338: 2273.0,
339: 2281.0,
340: 2287.0,
341: 2293.0,
342: 2297.0,
343: 2309.0,
344: 2311.0,
345: 2333.0,
346: 2339.0,

347: 2341.0,
348: 2347.0,
349: 2351.0,
350: 2357.0,
351: 2371.0,
352: 2377.0,
353: 2381.0,
354: 2383.0,
355: 2389.0,
356: 2393.0,
357: 2399.0,
358: 2411.0,
359: 2417.0,
360: 2423.0,
361: 2437.0,
362: 2441.0,
363: 2447.0,
364: 2459.0,
365: 2467.0,
366: 2473.0,
367: 2477.0,
368: 2503.0,
369: 2521.0,
370: 2531.0,
371: 2539.0,
372: 2543.0,
373: 2549.0,
374: 2551.0,
375: 2557.0,
376: 2579.0,
377: 2591.0,
378: 2593.0,
379: 2609.0,
380: 2617.0,
381: 2621.0,
382: 2633.0,
383: 2647.0,
384: 2657.0,
385: 2659.0,
386: 2663.0,
387: 2671.0,
388: 2677.0,
389: 2683.0,

390: 2687.0,
391: 2689.0,
392: 2693.0,
393: 2699.0,
394: 2707.0,
395: 2711.0,
396: 2713.0,
397: 2719.0,
398: 2729.0,
399: 2731.0,
400: 2741.0,
401: 2749.0,
402: 2753.0,
403: 2767.0,
404: 2777.0,
405: 2789.0,
406: 2791.0,
407: 2797.0,
408: 2801.0,
409: 2803.0,
410: 2819.0,
411: 2833.0,
412: 2837.0,
413: 2843.0,
414: 2851.0,
415: 2857.0,
416: 2861.0,
417: 2879.0,
418: 2887.0,
419: 2897.0,
420: 2903.0,
421: 2909.0,
422: 2917.0,
423: 2927.0,
424: 2939.0,
425: 2953.0,
426: 2957.0,
427: 2963.0,
428: 2969.0,
429: 2971.0,
430: 2999.0,
431: 3001.0,
432: 3011.0,

433: 3019.0,
434: 3023.0,
435: 3037.0,
436: 3041.0,
437: 3049.0,
438: 3061.0,
439: 3067.0,
440: 3079.0,
441: 3083.0,
442: 3089.0,
443: 3109.0,
444: 3119.0,
445: 3121.0,
446: 3137.0,
447: 3163.0,
448: 3167.0,
449: 3169.0,
450: 3181.0,
451: 3187.0,
452: 3191.0,
453: 3203.0,
454: 3209.0,
455: 3217.0,
456: 3221.0,
457: 3229.0,
458: 3251.0,
459: 3253.0,
460: 3257.0,
461: 3259.0,
462: 3271.0,
463: 3299.0,
464: 3301.0,
465: 3307.0,
466: 3313.0,
467: 3319.0,
468: 3323.0,
469: 3329.0,
470: 3331.0,
471: 3343.0,
472: 3347.0,
473: 3359.0,
474: 3361.0,
475: 3371.0,

476: 3373.0,
477: 3389.0,
478: 3391.0,
479: 3407.0,
480: 3413.0,
481: 3433.0,
482: 3449.0,
483: 3457.0,
484: 3461.0,
485: 3463.0,
486: 3467.0,
487: 3469.0,
488: 3491.0,
489: 3499.0,
490: 3511.0,
491: 3517.0,
492: 3527.0,
493: 3529.0,
494: 3533.0,
495: 3539.0,
496: 3541.0,
497: 3547.0,
498: 3557.0,
499: 3559.0,
500: 3571.0,
501: 3581.0,
502: 3583.0,
503: 3593.0,
504: 3607.0,
505: 3613.0,
506: 3617.0,
507: 3623.0,
508: 3631.0,
509: 3637.0,
510: 3643.0,
511: 3659.0,
512: 3671.0,
513: 3673.0,
514: 3677.0,
515: 3691.0,
516: 3697.0,
517: 3701.0,
518: 3709.0,

519: 3719.0,
520: 3727.0,
521: 3733.0,
522: 3739.0,
523: 3761.0,
524: 3767.0,
525: 3769.0,
526: 3779.0,
527: 3793.0,
528: 3797.0,
529: 3803.0,
530: 3821.0,
531: 3823.0,
532: 3833.0,
533: 3847.0,
534: 3851.0,
535: 3853.0,
536: 3863.0,
537: 3877.0,
538: 3881.0,
539: 3889.0,
540: 3907.0,
541: 3911.0,
542: 3917.0,
543: 3919.0,
544: 3923.0,
545: 3929.0,
546: 3931.0,
547: 3943.0,
548: 3947.0,
549: 3967.0,
550: 3989.0,
551: 4001.0,
552: 4003.0,
553: 4007.0,
554: 4013.0,
555: 4019.0,
556: 4021.0,
557: 4027.0,
558: 4049.0,
559: 4051.0,
560: 4057.0,
561: 4073.0,

562: 4079.0,
563: 4091.0,
564: 4093.0,
565: 4099.0,
566: 4111.0,
567: 4127.0,
568: 4129.0,
569: 4133.0,
570: 4139.0,
571: 4153.0,
572: 4157.0,
573: 4159.0,
574: 4177.0,
575: 4201.0,
576: 4211.0,
577: 4217.0,
578: 4219.0,
579: 4229.0,
580: 4231.0,
581: 4241.0,
582: 4243.0,
583: 4253.0,
584: 4259.0,
585: 4261.0,
586: 4271.0,
587: 4273.0,
588: 4283.0,
589: 4289.0,
590: 4297.0,
591: 4327.0,
592: 4337.0,
593: 4339.0,
594: 4349.0,
595: 4357.0,
596: 4363.0,
597: 4373.0,
598: 4391.0,
599: 4397.0,
600: 4409.0,
601: 4421.0,
602: 4423.0,
603: 4441.0,
604: 4447.0,

605: 4451.0,
606: 4457.0,
607: 4463.0,
608: 4481.0,
609: 4483.0,
610: 4493.0,
611: 4507.0,
612: 4513.0,
613: 4517.0,
614: 4519.0,
615: 4523.0,
616: 4547.0,
617: 4549.0,
618: 4561.0,
619: 4567.0,
620: 4583.0,
621: 4591.0,
622: 4597.0,
623: 4603.0,
624: 4621.0,
625: 4637.0,
626: 4639.0,
627: 4643.0,
628: 4649.0,
629: 4651.0,
630: 4657.0,
631: 4663.0,
632: 4673.0,
633: 4679.0,
634: 4691.0,
635: 4703.0,
636: 4721.0,
637: 4723.0,
638: 4729.0,
639: 4733.0,
640: 4751.0,
641: 4759.0,
642: 4783.0,
643: 4787.0,
644: 4789.0,
645: 4793.0,
646: 4799.0,
647: 4801.0,

648: 4813.0,
649: 4817.0,
650: 4831.0,
651: 4861.0,
652: 4871.0,
653: 4877.0,
654: 4889.0,
655: 4903.0,
656: 4909.0,
657: 4919.0,
658: 4931.0,
659: 4933.0,
660: 4937.0,
661: 4943.0,
662: 4951.0,
663: 4957.0,
664: 4967.0,
665: 4969.0,
666: 4973.0,
667: 4987.0,
668: 4993.0,
669: 4999.0,
670: 5003.0,
671: 5009.0,
672: 5011.0,
673: 5021.0,
674: 5023.0,
675: 5039.0,
676: 5051.0,
677: 5059.0,
678: 5077.0,
679: 5081.0,
680: 5087.0,
681: 5099.0,
682: 5101.0,
683: 5107.0,
684: 5113.0,
685: 5119.0,
686: 5147.0,
687: 5153.0,
688: 5167.0,
689: 5171.0,
690: 5179.0,

691: 5189.0,
692: 5197.0,
693: 5209.0,
694: 5227.0,
695: 5231.0,
696: 5233.0,
697: 5237.0,
698: 5261.0,
699: 5273.0,
700: 5279.0,
701: 5281.0,
702: 5297.0,
703: 5303.0,
704: 5309.0,
705: 5323.0,
706: 5333.0,
707: 5347.0,
708: 5351.0,
709: 5381.0,
710: 5387.0,
711: 5393.0,
712: 5399.0,
713: 5407.0,
714: 5413.0,
715: 5417.0,
716: 5419.0,
717: 5431.0,
718: 5437.0,
719: 5441.0,
720: 5443.0,
721: 5449.0,
722: 5471.0,
723: 5477.0,
724: 5479.0,
725: 5483.0,
726: 5501.0,
727: 5503.0,
728: 5507.0,
729: 5519.0,
730: 5521.0,
731: 5527.0,
732: 5531.0,
733: 5557.0,

734: 5563.0,
735: 5569.0,
736: 5573.0,
737: 5581.0,
738: 5591.0,
739: 5623.0,
740: 5639.0,
741: 5641.0,
742: 5647.0,
743: 5651.0,
744: 5653.0,
745: 5657.0,
746: 5659.0,
747: 5669.0,
748: 5683.0,
749: 5689.0,
750: 5693.0,
751: 5701.0,
752: 5711.0,
753: 5717.0,
754: 5737.0,
755: 5741.0,
756: 5743.0,
757: 5749.0,
758: 5779.0,
759: 5783.0,
760: 5791.0,
761: 5801.0,
762: 5807.0,
763: 5813.0,
764: 5821.0,
765: 5827.0,
766: 5839.0,
767: 5843.0,
768: 5849.0,
769: 5851.0,
770: 5857.0,
771: 5861.0,
772: 5867.0,
773: 5869.0,
774: 5879.0,
775: 5881.0,
776: 5897.0,

777: 5903.0,
778: 5923.0,
779: 5927.0,
780: 5939.0,
781: 5953.0,
782: 5981.0,
783: 5987.0,
784: 6007.0,
785: 6011.0,
786: 6029.0,
787: 6037.0,
788: 6043.0,
789: 6047.0,
790: 6053.0,
791: 6067.0,
792: 6073.0,
793: 6079.0,
794: 6089.0,
795: 6091.0,
796: 6101.0,
797: 6113.0,
798: 6121.0,
799: 6131.0,
800: 6133.0,
801: 6143.0,
802: 6151.0,
803: 6163.0,
804: 6173.0,
805: 6197.0,
806: 6199.0,
807: 6203.0,
808: 6211.0,
809: 6217.0,
810: 6221.0,
811: 6229.0,
812: 6247.0,
813: 6257.0,
814: 6263.0,
815: 6269.0,
816: 6271.0,
817: 6277.0,
818: 6287.0,
819: 6299.0,

820: 6301.0,
821: 6311.0,
822: 6317.0,
823: 6323.0,
824: 6329.0,
825: 6337.0,
826: 6343.0,
827: 6353.0,
828: 6359.0,
829: 6361.0,
830: 6367.0,
831: 6373.0,
832: 6379.0,
833: 6389.0,
834: 6397.0,
835: 6421.0,
836: 6427.0,
837: 6449.0,
838: 6451.0,
839: 6469.0,
840: 6473.0,
841: 6481.0,
842: 6491.0,
843: 6521.0,
844: 6529.0,
845: 6547.0,
846: 6551.0,
847: 6553.0,
848: 6563.0,
849: 6569.0,
850: 6571.0,
851: 6577.0,
852: 6581.0,
853: 6599.0,
854: 6607.0,
855: 6619.0,
856: 6637.0,
857: 6653.0,
858: 6659.0,
859: 6661.0,
860: 6673.0,
861: 6679.0,
862: 6689.0,

863: 6691.0,
864: 6701.0,
865: 6703.0,
866: 6709.0,
867: 6719.0,
868: 6733.0,
869: 6737.0,
870: 6761.0,
871: 6763.0,
872: 6779.0,
873: 6781.0,
874: 6791.0,
875: 6793.0,
876: 6803.0,
877: 6823.0,
878: 6827.0,
879: 6829.0,
880: 6833.0,
881: 6841.0,
882: 6857.0,
883: 6863.0,
884: 6869.0,
885: 6871.0,
886: 6883.0,
887: 6899.0,
888: 6907.0,
889: 6911.0,
890: 6917.0,
891: 6947.0,
892: 6949.0,
893: 6959.0,
894: 6961.0,
895: 6967.0,
896: 6971.0,
897: 6977.0,
898: 6983.0,
899: 6991.0,
900: 6997.0,
901: 7001.0,
902: 7013.0,
903: 7019.0,
904: 7027.0,
905: 7039.0,

906: 7043.0,
907: 7057.0,
908: 7069.0,
909: 7079.0,
910: 7103.0,
911: 7109.0,
912: 7121.0,
913: 7127.0,
914: 7129.0,
915: 7151.0,
916: 7159.0,
917: 7177.0,
918: 7187.0,
919: 7193.0,
920: 7207.0,
921: 7211.0,
922: 7213.0,
923: 7219.0,
924: 7229.0,
925: 7237.0,
926: 7243.0,
927: 7247.0,
928: 7253.0,
929: 7283.0,
930: 7297.0,
931: 7307.0,
932: 7309.0,
933: 7321.0,
934: 7331.0,
935: 7333.0,
936: 7349.0,
937: 7351.0,
938: 7369.0,
939: 7393.0,
940: 7411.0,
941: 7417.0,
942: 7433.0,
943: 7451.0,
944: 7457.0,
945: 7459.0,
946: 7477.0,
947: 7481.0,
948: 7487.0,

949: 7489.0,
950: 7499.0,
951: 7507.0,
952: 7517.0,
953: 7523.0,
954: 7529.0,
955: 7537.0,
956: 7541.0,
957: 7547.0,
958: 7549.0,
959: 7559.0,
960: 7561.0,
961: 7573.0,
962: 7577.0,
963: 7583.0,
964: 7589.0,
965: 7591.0,
966: 7603.0,
967: 7607.0,
968: 7621.0,
969: 7639.0,
970: 7643.0,
971: 7649.0,
972: 7669.0,
973: 7673.0,
974: 7681.0,
975: 7687.0,
976: 7691.0,
977: 7699.0,
978: 7703.0,
979: 7717.0,
980: 7723.0,
981: 7727.0,
982: 7741.0,
983: 7753.0,
984: 7757.0,
985: 7759.0,
986: 7789.0,
987: 7793.0,
988: 7817.0,
989: 7823.0,
990: 7829.0,
991: 7841.0,

```

992: 7853.0,
993: 7867.0,
994: 7873.0,
995: 7877.0,
996: 7879.0,
997: 7883.0,
998: 7901.0,
999: 7907.0}

```

```

def run_all_setups_primes(X, y):
    jl.seval(
        """
        function p(i::T) where T
            if 0.5 < i < 1000
                return T(prime(round(Int, i)))
            else
                return T(NaN)
            end
        end
        """
    )
    class sympy_p(sympy.Function):
        pass
    print("KONFIGURACJA 1")
    print("  binary_operators: ['+', '*'], unary_operators: ['cos', 'exp',
↪  'sin', 'p']")

    model1 = PySRRegressor(
        binary_operators=["+", "*"],
        unary_operators=["cos", "exp", "sin", "p"],
        maxsize=20,
        extra_sympy_mappings={"p": sympy_p},
        parallelism="multithreading",
        **default_pysr_params,
    )
    with warnings.catch_warnings():
        warnings.filterwarnings(
            "ignore",
            message=r"Note: it looks like you are running in Jupyter.*",
            category=UserWarning,
        )
    model1.fit(X, y)

```

```

print("\nTop 3 rozwiązania (maksymalny score):")
eq1 = model1.equations_
top3_eq1 = eq1.nlargest(3, 'score')[['complexity', 'loss', 'score',
↪ 'equation']]
for idx, row in top3_eq1.iterrows():
    print(f"    [{row['complexity']}] Score: {row['score']:.6f}, Loss:
↪ {row['loss']:.6f}")
    print(f"    Równanie: {row['equation']}")

print("\nBest solution (model_selection='best'):")
best1 = model1.sympy()
print(f"    {best1}")

print("\nKONFIGURACJA 2")
print("    binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos',
↪ 'exp', 'sin', 'log', 'p']")

model2 = PySRRegressor(
    binary_operators=["+", "*", "-", "^"],
    unary_operators=["cos", "exp", "sin", "log", "p"],
    maxsize=30,
    constraints={"^": (-1, 1)},
    extra_sympy_mappings={"p": sympy_p},
    parallelism="multithreading",
    **default_pysr_params,
)
with warnings.catch_warnings():
    warnings.filterwarnings(
        "ignore",
        message=r"Note: it looks like you are running in Jupyter.*",
        category=UserWarning,
    )
    model2.fit(X, y)

print("\nTop 3 rozwiązania (maksymalny score):")
eq2 = model2.equations_
top3_eq2 = eq2.nlargest(3, 'score')[['complexity', 'loss', 'score',
↪ 'equation']]
for idx, row in top3_eq2.iterrows():
    print(f"    [{row['complexity']}] Score: {row['score']:.6f}, Loss:
↪ {row['loss']:.6f}")

```

```

        print(f" Równanie: {row['equation']}")

print("\nBest solution (model_selection='best'):")
best2 = model2.sympy()
print(f" {best2}")

# Konfiguracja 3
print("\nKONFIGURACJA 3")
print(" binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp',
↪ 'sin', 'p']")

model3 = PySRRegressor(
    binary_operators=["+", "*", "-", "^"],
    unary_operators=["exp", "sin", "p"],
    maxsize=15,
    constraints={"^": (-1, 1)},
    extra_sympy_mappings={"p": sympy_p},
    parallelism="multithreading",
    **default_pysr_params,
)
with warnings.catch_warnings():
    warnings.filterwarnings(
        "ignore",
        message=r"Note: it looks like you are running in Jupyter.*",
        category=UserWarning,
    )
    model3.fit(X, y)

print("\nTop 3 rozwiązania (maksymalny score):")
eq3 = model3.equations_
top3_eq3 = eq3.nlargest(3, 'score')[['complexity', 'loss', 'score',
↪ 'equation']]
for idx, row in top3_eq3.iterrows():
    print(f" [{row['complexity']}] Score: {row['score']:.6f}, Loss:
↪ {row['loss']:.6f}")
    print(f" Równanie: {row['equation']}")

print("\nBest solution (model_selection='best'):")
best3 = model3.sympy()
print(f" {best3}")

return model1, model2, model3

```

Liczby pierwsze są zwracane tylko dla indeksów większych od zera, dlatego zmieniony został zakres X z -5 do 5 na 1 do 11 oraz z -15 do 15 na 1-31

```
X = np.random.uniform(1, 11, size=(200,6))

y_primes = 2.2 * np.sin(X[:, 0] + 2 * X[:, 1]) - X[:, 5] ** 2 - 3 -
↳ np.array([primes[int(np.floor(np.abs(x)))] for x in X[:, 0]])
```

```
np.random.seed(0)

noise = np.random.normal(0, 0.5, 200)
y_primes_noisy = 2.2 * np.sin(X[:, 0] + 2 * X[:, 1]) - X[:, 5] ** 2 - 3 -
↳ np.array([primes[int(np.floor(np.abs(x)))] for x in X[:, 0]]) + noise
```

```
np.random.seed(0)
X_15 = np.random.uniform(1, 31, size=(200,6))
y_primes_15_no_noise = 2.2 * np.sin(X_15[:, 0] + 2 * X_15[:, 1]) - X_15[:, 5]
↳ ** 2 - 3 - np.array([primes[int(np.floor(np.abs(x)))] for x in X_15[:,
↳ 0]])

noise0_5 = np.random.normal(0, 0.5, 200)
y_primes_noisy0_5 = y_primes_15_no_noise + noise0_5

noise2 = np.random.normal(0, 2, 200)
y_primes_noisy2 = y_primes_15_no_noise + noise2

noise5 = np.random.normal(0, 5, 200)
y_primes_noisy5 = y_primes_15_no_noise + noise5
```

Zostały powtórzone eksperymenty:

0. od 1 - 11 bez szumu
1. od 1 - 11 z szumem o std = 0.5
2. od 1 - 31 bez szumu
3. od 1 - 31 z szumem o std = 0.5
4. od 1 - 31 z szumem o std = 2.0
5. od 1 - 31 z szumem o std = 5.0

```
print("Próby na próbce bez szumu (z liczbami pierwszymi)")
model1_primes_0, model2_primes_0, model3_primes_0 = run_all_setups_primes(X,
↳ y_primes)
```

Próby na próbce bez szumu (z liczbami pierwszymi)

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']

Top 3 rozwiązania (maksymalny score):

[9] Score: 1.996577, Loss: 5.365956

Równanie: $(x_0 * -2.758316) + ((x_5 * -0.9890287) * x_5)$

[3] Score: 1.099794, Loss: 155.257490

Równanie: $x_5 * -10.623044$

[8] Score: 0.529120, Loss: 39.513870

Równanie: $(x_0 + (p(x_5) + -4.9006968)) * -3.8687196$

Best solution (model_selection='best'):

$x_5 * x_5 * (-0.9934681) + \text{sympy_p}(x_0 + 1.4936566) * (-0.8403533)$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log', 'p']

Top 3 rozwiązania (maksymalny score):

[28] Score: 10.415038, Loss: 0.000000

Równanie: $(((((\sin(x_1 + (x_0 + x_1)) * 1.1999997) + \sin(x_1 + (x_0 + x_1))) -$
↪ $(x_5 * x_5)) - 0.8148504) - p(x_0 + -0.50558275)) - 2.1851497$

[7] Score: 2.511128, Loss: 5.707288

Równanie: $(x_0 * -2.7440262) - (x_5 * x_5)$

[17] Score: 1.179580, Loss: 0.713724

Równanie: $((\sin(x_0 + (x_1 + x_1)) - p(x_0 + -0.5047128)) - (x_5 * x_5)) -$
↪ 2.9085026

Best solution (model_selection='best'):

$-x_5 * x_5 - \text{sympy_p}(x_0 - 0.50558275) + \sin(x_0 + x_1 + x_1) + \sin(x_0 + x_1 +$
↪ $x_1) * 1.1999997 - 1 * 2.1851497 - 1 * 0.8148504$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin',
↪ 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 2.482023, Loss: 5.569418

Równanie: $(x_0 * -2.6908677) - (x_5 * x_5)$

[3] Score: 1.099794, Loss: 155.257490

Równanie: $x_5 * -10.623024$

[9] Score: 0.371383, Loss: 3.257717

Równanie: $-1.8117535 - ((x_0 \wedge 1.4254878) + (x_5 \wedge 2.0000088))$

```
Best solution (model_selection='best'):
-(x0**1.4254878 + x5**2.0000088) - 1.8117535
```

```
print("Próby na próbce z szumem o std=0.5 (z liczbami pierwszymi)")
model1_primes_1, model2_primes_1, model3_primes_1 = run_all_setups_primes(X,
↪ y_primes_noisy)
```

Próby na próbce z szumem o std=0.5 (z liczbami pierwszymi)

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']

Top 3 rozwiązania (maksymalny score):

[8] Score: 2.552797, Loss: 5.274408
Równanie: $(p(x_0) + (x_5 * x_5)) * -1.0121$
[3] Score: 1.099321, Loss: 154.997830
Równanie: $x_5 * -10.615126$
[4] Score: 0.327706, Loss: 111.687500
Równanie: $p(x_5) * -4.1666303$

Best solution (model_selection='best'):

$(x_5 * x_5 + \text{sympy_p}(x_0 - 0.50530493) + 3.0143988) * (-0.9965977)$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log', 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 2.426191, Loss: 5.874407
Równanie: $(x_0 * -2.6866298) - (x_5 * x_5)$
[3] Score: 1.099321, Loss: 154.997850
Równanie: $x_5 * -10.615065$
[22] Score: 0.762570, Loss: 0.256154
Równanie: $(-9.99866 - p(x_0 + -0.50522286)) + ((\cos(\cos((0.80533534 + (x_0 *
↪ 0.4984074)) + x_1)) * 9.200807) - (x_5 * x_5))$

Best solution (model_selection='best'):

$-x_5 * x_5 - \text{sympy_p}(x_0 - 0.50522286) + \cos(\cos(x_0 * 0.4984074 + x_1 +
↪ 0.80533534)) * 9.200807 - 9.99866$

KONFIGURACJA 3

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin',  
↪ 'p']
```

Top 3 rozwiązania (maksymalny score):

```
[7] Score: 2.426191, Loss: 5.874407  
Równanie: (x0 * -2.6866286) - (x5 * x5)  
[3] Score: 1.099321, Loss: 154.997850  
Równanie: x5 * -10.615059  
[10] Score: 0.336388, Loss: 2.481019  
Równanie: -2.7967029 - (p(x0 + -0.50561845) + (x5 * x5))
```

Best solution (model_selection='best'):

```
-(x5*x5 + sympy_p(x0 - 0.50561845)) - 2.7967029
```

```
print("Próby na próbce bez szumu od 1 do 31 (z liczbami pierwszymi)")  
model1_primes_2, model2_primes_2, model3_primes_2 =  
↪ run_all_setups_primes(X_15, y_primes_15_no_noise)
```

Próby na próbce bez szumu od 1 do 31 (z liczbami pierwszymi)

KONFIGURACJA 1

```
binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']
```

Top 3 rozwiązania (maksymalny score):

```
[8] Score: 4.798061, Loss: 10.827063  
Równanie: ((x5 * x5) + p(x0)) * -1.0016857  
[3] Score: 1.052152, Loss: 10760.169000  
Równanie: x5 * -26.123981  
[4] Score: 0.741447, Loss: 5126.401400  
Równanie: p(x5) * -7.3121276
```

Best solution (model_selection='best'):

```
(x5*x5 + sympy_p(x0 - 0.49571797) + 3.056533)*(-1.0000775)
```

KONFIGURACJA 2

```
binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',  
↪ 'sin', 'log', 'p']
```

Top 3 rozwiązania (maksymalny score):

```
[8] Score: 4.336962, Loss: 10.546574  
Równanie: (-0.986969 - (x5 * x5)) - p(x0)  
[23] Score: 3.859121, Loss: 0.000002
```

Równanie: $((((x_5 * x_5) - (\sin(x_0 + (x_1 * 2.0)) * 2.2004852)) + 1.5188272) * -0.999996) - (p(x_0 - 0.5012596) + 1.4823489)$
 ↳ [21] Score: 2.666721, Loss: 0.003392
 Równanie: $((((x_5 * x_5) - (\sin(x_0 + (x_1 * 2.0000865)) * 2.1995637)) * -1.0000049) - (p(x_0 - 0.50141007) + 2.940042)$
 ↳

Best solution (model_selection='best'):
 $-(\text{sympy_p}(x_0 - 1*0.5012596) + 1.4823489) + (x_5*x_5 - 2.2004852*\sin(x_0 + x_1*2.0) + 1.5188272)*(-0.999996)$
 ↳

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin', 'p']
 ↳

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.839273, Loss: 35.472150
 Równanie: $(x_0 * -3.6462846) - (x_5 * x_5)$
 [5] Score: 1.294866, Loss: 1404.298600
 Równanie: $-52.766827 - (x_5 * x_5)$
 [8] Score: 1.212947, Loss: 10.546573
 Równanie: $(-0.98679006 - (x_5 * x_5)) - p(x_0)$

Best solution (model_selection='best'):
 $-x_5*x_5 - \text{sympy_p}(x_0 - 1*0.5016944) - 3.1124175$

```
print("Próby na próbce z szumem o std=0.5 od 1 do 31 (z liczbami
↳ pierwszymi)")
model1_primes_3, model2_primes_3, model3_primes_3 =
↳ run_all_setups_primes(X_15, y_primes_noisy0_5)
```

Próby na próbce z szumem o std=0.5 od 1 do 31 (z liczbami pierwszymi)

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']

Top 3 rozwiązania (maksymalny score):

[8] Score: 4.777078, Loss: 11.035247
 Równanie: $((x_5 * x_5) + p(x_0)) * -1.001606$
 [3] Score: 1.051974, Loss: 10765.534000
 Równanie: $x_5 * -26.120617$
 [4] Score: 0.741162, Loss: 5130.420400
 Równanie: $p(x_5) * -7.311493$

Best solution (model_selection='best'):

$(x_5 * x_5 + \text{sympy_p}(x_0 - 0.49339408)) * (-1.0001456) - 2.959312$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log', 'p']

Top 3 rozwiązania (maksymalny score):

[8] Score: 4.312017, Loss: 10.814801

Równanie: $(-0.92178595 - p(x_0)) - (x_5 * x_5)$

[5] Score: 1.295574, Loss: 1404.404800

Równanie: $-52.702267 - (x_5 * x_5)$

[3] Score: 1.051974, Loss: 10765.534000

Równanie: $x_5 * -26.120584$

Best solution (model_selection='best'):

$-x_5 * x_5 - (\text{sympy_p}(x_0 - 1 * 0.50128746) + \cos(x_0 + x_1 + x_1 +$
↪ $1.5906271) * 2.217821) - 2.9345424$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin',
↪ 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.831828, Loss: 36.006973

Równanie: $(x_0 * -3.6430643) - (x_5 * x_5)$

[5] Score: 1.295574, Loss: 1404.404800

Równanie: $-52.7008 - (x_5 * x_5)$

[8] Score: 1.202797, Loss: 10.814800

Równanie: $(-0.9212972 - p(x_0)) - (x_5 * x_5)$

Best solution (model_selection='best'):

$-x_5 * x_5 - \text{sympy_p}(x_0 - 1 * 0.4966411) - 3.0166628$

```
print("Próby na próbce z szumem o std=2.0 od 1 do 31 (z liczbami  
↪ pierwszymi)")  
model1_primes_4, model2_primes_4, model3_primes_4 =  
↪ run_all_setups_primes(X_15, y_primes_noisy2)
```

Próby na próbce z szumem o std=2.0 od 1 do 31 (z liczbami pierwszymi)

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']

Top 3 rozwiązania (maksymalny score):

[8] Score: 4.441148, Loss: 15.210650
Równanie: $(p(x_0) + (x_5 * x_5)) * -1.001774$
[3] Score: 1.051804, Loss: 10779.365000
Równanie: $x_5 * -26.124378$
[4] Score: 0.742791, Loss: 5128.649000
Równanie: $p(x_5) * -7.3128314$

Best solution (model_selection='best'):

$(x_5 * x_5 + \text{sympy_p}(x_0 - 0.4689734)) * (-1.0049387)$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log', 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.797256, Loss: 38.058310
Równanie: $(x_0 * -3.6338742) - (x_5 * x_5)$
[3] Score: 1.051804, Loss: 10779.365000
Równanie: $x_5 * -26.124378$
[5] Score: 1.025879, Loss: 1385.243400
Równanie: $-52.700863 - (x_5 * x_5)$

Best solution (model_selection='best'):

$-(x_5 * x_5 + 2.931462) - \text{sympy_p}(x_0 - 0.5055686) + \sin(x_0 + x_1 + x_1)$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin',
↪ 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.797256, Loss: 38.058310
Równanie: $(x_0 * -3.6338742) - (x_5 * x_5)$
[5] Score: 1.308966, Loss: 1385.243700
Równanie: $-52.70452 - (x_5 * x_5)$
[3] Score: 1.051804, Loss: 10779.365000
Równanie: $x_5 * -26.124367$

Best solution (model_selection='best'):

$-x_5 * x_5 - \text{sympy_p}(x_0 - 0.5053993) - 3.0582042$

```
print("Próby na próbce z szumem o std=5.0 od 1 do 31 (z liczbami
↪ pierwszymi)")
model1_primes_5, model2_primes_5, model3_primes_5 =
↪ run_all_setups_primes(X_15, y_primes_noisy5)
```

Próby na próbce z szumem o std=5.0 od 1 do 31 (z liczbami pierwszymi)

KONFIGURACJA 1

binary_operators: ['+', '*'], unary_operators: ['cos', 'exp', 'sin', 'p']

Top 3 rozwiązania (maksymalny score):

[8] Score: 3.630648, Loss: 36.187065
Równanie: $(p(x_0) + (x_5 * x_5)) * -1.001645$
[3] Score: 1.052502, Loss: 10754.937500
Równanie: $x_5 * -26.123657$
[4] Score: 0.739941, Loss: 5131.632300
Równanie: $p(x_5) * -7.3121476$

Best solution (model_selection='best'):

$(x_5 * x_5 + \text{sympy_p}(x_0 - 0.4000445)) * (-1.0044172)$

KONFIGURACJA 2

binary_operators: ['+', '*', '-', '^'], unary_operators: ['cos', 'exp',
↪ 'sin', 'log', 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.621508, Loss: 57.252785
Równanie: $(x_0 * -3.6630814) - (x_5 * x_5)$
[5] Score: 1.252687, Loss: 1466.292500
Równanie: $-52.75025 - (x_5 * x_5)$
[3] Score: 1.052502, Loss: 10754.937500
Równanie: $x_5 * -26.123657$

Best solution (model_selection='best'):

$-x_5 * x_5 - \text{sympy_p}(x_0 - 0.47259942) - 2.9651413$

KONFIGURACJA 3

binary_operators: ['+', '*', '-', '^'], unary_operators: ['exp', 'sin',
↪ 'p']

Top 3 rozwiązania (maksymalny score):

[7] Score: 1.621508, Loss: 57.252777
Równanie: $(x_0 * -3.6630898) - (x_5 * x_5)$

```
[5] Score: 1.252687, Loss: 1466.292500  
Równanie:  $-52.748993 - (x_5 * x_5)$   
[3] Score: 1.052502, Loss: 10754.937000  
Równanie:  $x_5 * -26.123507$ 
```

```
Best solution (model_selection='best'):  
-x5*x5 - sympy_p(x0) - 0.9698069
```